

SINAI UNIVERSITY

Faculty of Information Technology and Computer Science

Department of Information Technology

PROJECT DOCUMENTATION

Cairo Metro Route Planner

*An ASP.NET Core MVC Application for Shortest Path Calculation
and Fare Estimation in the Cairo Metro Network*

PREPARED BY

Abd Alrahman Saed Alsayed — 202302377

Mahmoud Mohamed Hegazy — 202302612

Mahmoud Fawzy Mahmoud — 202302484

Ali Yasser Mohamed — 202305666

Abdullah Ibrahiem Gaballah — 202302701

SUPERVISED BY

Dr. Hossam Eladly

Academic Year 2025 / 2026

Submission Date: June 2026

Table of Contents

1 Executive Summary

2 Introduction

3 System Architecture

3.1 Presentation Layer

3.2 Business Logic Layer

3.3 Data Access Layer

3.4 Database Layer

4 Technology Stack

5 Database Design

5.1 Entity Relationship Diagram

5.2 Table Definitions

5.3 Relationships

5.4 Normalization

6 Entity Framework Core

6.1 DbContext

6.2 Fluent API Configurations

6.3 Repositories

6.4 Data Seeding

7 Business Logic Layer

7.1 Graph Construction

7.2 Shortest Path Algorithm

7.3 Pricing Calculation

7.4 Travel Time Estimation

7.5 Transfer Detection

7.6 Service Orchestration

8 Controllers

8.1 RoutesController

8.2 HomeController

9 Views and User Interface

10 Security

11 Performance Considerations

12 Testing and Validation

13 Challenges and Solutions

14 Future Enhancements

15 Conclusion

1. Executive Summary

The Cairo Metro Route Planner is a web-based application developed using ASP.NET Core MVC that addresses the real-world problem of navigating the Cairo Metro network. With three operational metro lines, eighty-nine stations, and five transfer points, passengers face significant difficulty in determining optimal routes, calculating fares, and estimating travel times.

This system provides an interactive solution that computes the shortest path between any two stations using Dijkstra's algorithm, calculates the applicable fare based on a tiered pricing structure, estimates travel time, and detects the number of line transfers required. The application features a searchable station selection interface using Tom Select, an interactive map rendered with Leaflet and OpenStreetMap tiles, and a GPS-based nearest station locator using the Haversine formula.

The application is built on a three-tier architecture comprising a presentation layer (Metro), a business logic layer (Metro.Core), and a data access layer (Metro.Data). The technology stack includes .NET 8.0, Entity Framework Core 8.0, and SQL Server. The system follows the MVC architectural pattern, the Repository pattern for data access, and Dependency Injection for loose coupling between components.

Key features include Dijkstra's algorithm with a priority queue for optimal path finding, memory-cached graph construction for performance, a tiered pricing engine, automated transfer detection, interactive mapping capabilities, and comprehensive client-side and server-side validation.

2. Introduction

2.1 Background

The Cairo Metro is one of the busiest rapid transit systems in Africa and the Middle East, serving millions of passengers daily across three interconnected lines. The network spans from Helwan in the south to El-Marg in the north (Line 1), from Shubra El-Kheima to El-Mounib (Line 2), and from Adly Mansour to Cairo University (Line 3). With eighty-nine stations and multiple intersecting transfer points, navigating the system efficiently requires computational assistance.

2.2 Objectives

The primary objectives of this project are as follows:

- To design and implement a web application that calculates the shortest path between any two metro stations in the Cairo Metro network.
- To provide accurate fare estimation based on a tiered pricing structure defined by station count.
- To estimate travel time using a standardized calculation of two minutes per station traversed.
- To detect and report the number of line transfers required along a given route.
- To deliver an intuitive, interactive user interface with mapping and GPS functionality.
- To apply software engineering best practices including the MVC pattern, Dependency Injection, the Repository pattern, and layered architecture.

2.3 Scope

The scope of this project encompasses the development of a complete three-tier web application. The system covers the entire Cairo Metro network as of the 2025-2026 period, including all three lines, their stations, and interconnections. The application provides route calculation, fare estimation, travel time estimation, transfer detection, and interactive mapping. The system does not cover real-time train schedules, live occupancy data, or mobile native application development.

2.4 System Overview

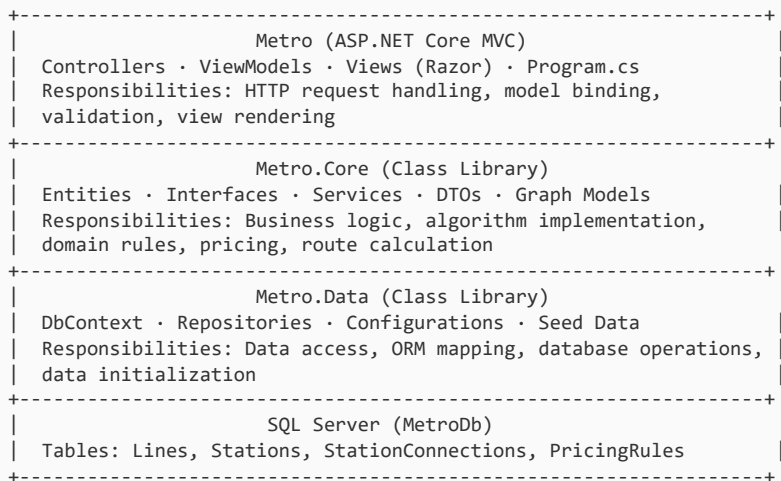
The system is organized into three distinct projects within a single solution. The **Metro** project serves as the presentation layer, containing ASP.NET Core MVC controllers, view models, and Razor views. The **Metro.Core** project contains the business logic layer with domain entities, service interfaces and implementations, data transfer objects (DTOs), and graph-related models. The **Metro.Data** project provides the data access layer, including the Entity Framework Core DbContext, entity configurations, repository implementations, and JSON-based data seeding.

The system accepts user input in the form of a departure station and a destination station selected from searchable dropdown lists. Upon submission, the system invokes a chain of services: the orchestrator service delegates to the route service for path computation, the pricing service for fare calculation, the travel time service for duration estimation, and the transfer detection service for counting line changes. The result is presented to the user through a structured result card displaying station count, fare, estimated time, number of transfers, and a sequential list of stations along the path.

3. System Architecture

3.1 Overall Architecture

The Cairo Metro Route Planner follows a three-tier (layered) architecture that separates concerns across three logical layers: presentation, business logic, and data access. This architectural style promotes maintainability, testability, and separation of concerns. Each layer has a distinct responsibility and communicates with adjacent layers through well-defined interfaces.



3.2 Presentation Layer (Metro)

The presentation layer is implemented as an ASP.NET Core MVC web application. It contains two controllers: **RoutesController**, which handles route search requests using strongly-typed view models and the orchestrator service, and **HomeController**, which serves the landing page and an older implementation of route search using ViewBag. The layer also contains view models (**RouteSearchViewModel**, **StationOptionViewModel**, **ErrorViewModel**) and Razor views (**Index.cshtml** for both controllers, a shared layout, error page, and supporting partial views).

3.3 Business Logic Layer (Metro.Core)

The business logic layer contains the core domain entities (**Station**, **Line**, **StationConnection**, **PricingRule**), service interfaces and implementations, data transfer objects, and graph-related models. The six primary services are:

- **GraphBuilder** — Constructs an adjacency dictionary representing the metro network graph from database records and caches it in memory for thirty minutes.
- **RouteService** — Implements Dijkstra's shortest path algorithm using a priority queue (min-heap).
- **PricingService** — Calculates fares based on a tiered pricing structure matched against station count.
- **TravelTimeService** — Estimates travel time using a fixed rate of two minutes per station.

- **TransferDetectionService** — Counts line transfers by comparing consecutive station LineId values along the path.
- **MetroService** — Orchestrator service that coordinates the five specialist services and resolves station entities.

3.4 Data Access Layer (Metro.Data)

The data access layer provides the Entity Framework Core DbContext (**MetroDbContext**), four repository implementations (**StationRepository**, **StationConnectionRepository**, **PricingRuleRepository**, **LineRepository**), Fluent API entity configurations, and a data seeding mechanism that reads JSON files from the file system. Repositories abstract the underlying data storage and provide a clean interface for the business logic layer to query and persist data.

3.5 Database Layer (SQL Server)

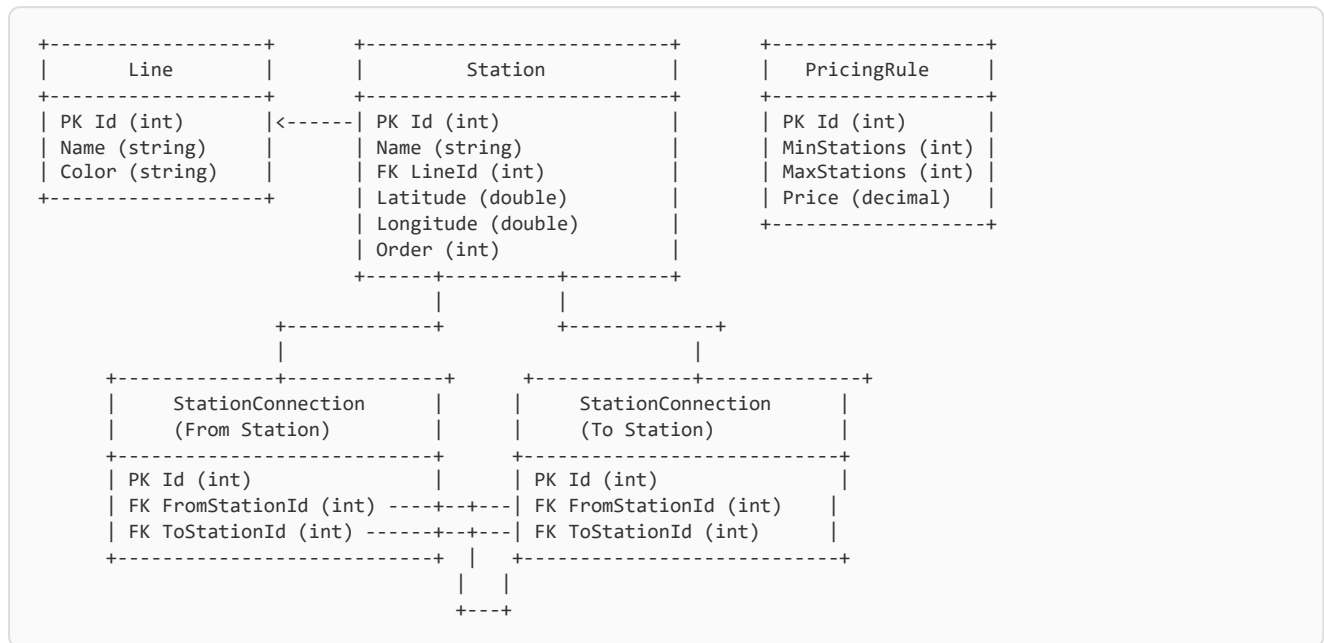
The database layer consists of a SQL Server database named **MetroDb** with four tables: Lines, Stations, StationConnections, and PricingRules. The database is accessed exclusively through Entity Framework Core, which maps the relational tables to the domain entities defined in Metro.Core.

4. Technology Stack

Technology	Version	Purpose	Rationale
ASP.NET Core MVC	8.0	Web application framework	Provides a mature, industry-standard framework for building web applications with the MVC pattern. Supports dependency injection, model binding, validation, and Razor view engine natively.
Entity Framework Core	8.0	Object-relational mapping	Offers a lightweight, extensible ORM with LINQ integration, migrations, and a Fluent API for fine-grained entity configuration. Enables database-agnostic data access.
SQL Server	—	Relational database	Selected for its robust relational capabilities, wide adoption in enterprise environments, and seamless integration with Entity Framework Core through the SqlServer provider.
Dependency Injection	Built-in	Loose coupling, testability	ASP.NET Core includes a built-in IoC container. All services and repositories are registered as scoped services and injected through constructor injection.
LINQ	—	Data querying	Language-Integrated Query enables type-safe, compile-time-checked queries against the database. Used in all repositories for filtering, projection, and ordering.
Bootstrap 5	5.x	CSS framework	Provides responsive grid system, pre-built components, and consistent styling across devices.
Tom Select	Latest	Searchable dropdowns	Transforms standard HTML select elements into searchable, user-friendly dropdowns. Essential for usability given eighty-nine stations in the network.
Leaflet + OpenStreetMap	Latest	Interactive maps	Lightweight, open-source mapping library that renders interactive maps with OpenStreetMap tiles. Supports markers, popups, and GPS integration.
jQuery	3.x	Client-side scripting	Simplifies DOM manipulation, event handling, and AJAX operations.
xUnit	2.9.3	Unit testing	Industry-standard testing framework for .NET. Used with Moq for mocking dependencies.

5. Database Design

5.1 Entity Relationship Diagram



5.2 Table Definitions

Lines — Stores the three metro lines.

Column	Type	Constraints	Description
Id	int	Primary Key, ValueGeneratedNever	Unique identifier for the line
Name	nvarchar(100)	Required	Line name (e.g., "Line 1")
Color	nvarchar(50)	Required	Display color (e.g., "Red")

Stations — Stores the eighty-nine metro stations.

Column	Type	Constraints	Description
Id	int	Primary Key, ValueGeneratedNever	Unique identifier for the station
Name	nvarchar(max)	—	Station name
LineId	int	Foreign Key → Lines.Id, DeleteBehavior.Restrict	The line this station belongs to
Latitude	float	—	Geographic latitude coordinate
Longitude	float	—	Geographic longitude coordinate
Order	int	—	Sequential position on the line

StationConnections — Stores directed connections between adjacent stations.

Column	Type	Constraints	Description
Id	int	Primary Key, ValueGeneratedNever	Unique identifier
FromStationId	int	Foreign Key → Stations.Id, Restrict	Source station
ToStationId	int	Foreign Key → Stations.Id, Restrict	Destination station

PricingRules — Stores fare pricing tiers.

Column	Type	Constraints	Description
Id	int	Primary Key, ValueGeneratedNever	Unique identifier
MinStations	int	—	Minimum stations for this tier
MaxStations	int	—	Maximum stations for this tier
Price	decimal	—	Fare in Egyptian Pounds

5.3 Relationships

Relationship	Type	Description
Line → Station	One-to-Many	A single line contains multiple stations. Each station belongs to exactly one line. Implemented via Station.LineId foreign key.
Station → StationConnection (From)	One-to-Many	A station can be the source of multiple connections. Implemented via StationConnection.FromStationId.
Station → StationConnection (To)	One-to-Many	A station can be the destination of multiple connections. Implemented via StationConnection.ToStationId.

5.4 Normalization

The database schema conforms to Third Normal Form (3NF). Each table represents a single entity type. Station data is not repeated across lines; instead, a LineId foreign key references the Lines table. Connection data is stored in a separate table rather than embedded within station records, eliminating redundancy and ensuring referential integrity. Pricing rules are stored in a dedicated table with tier boundaries, avoiding hard-coded values in application logic.

6. Entity Framework Core

6.1 DbContext

The **MetroDbContext** class extends **DbContext** and serves as the bridge between the application code and the SQL Server database. It exposes four **DbSet** properties representing each table in the database, and overrides the **OnModelCreating** method to apply entity configurations discovered via reflection.

```
public class MetroDbContext : DbContext
{
    [..]
    [..]
    [..]
    public DbSet<PricingRule> PricingRules { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        [..]
        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(MetroDbContext).Assembly);
        base.OnModelCreating(modelBuilder);
    }
}
```

The **ApplyConfigurationsFromAssembly** method automatically discovers and applies all classes implementing **IEntityTypeConfiguration<T>** within the same assembly, eliminating the need for manual registration of each configuration.

The **DbContext** is registered in the dependency injection container using **AddDbContextPool**, which enables context pooling for improved performance under load:

```
[..]
builder.Services.AddDbContextPool<MetroDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));
```

6.2 Fluent API Configurations

StationConfiguration:

- Primary key on **Id** with **ValueGeneratedNever()** — identifiers are supplied from seed data rather than auto-generated by the database.
- A composite index on (**LineId**, **Order**) to optimize queries that retrieve stations of a specific line in order.
- A required foreign key relationship to **Line** with **DeleteBehavior.Restrict** to prevent accidental cascade deletes.

LineConfiguration:

- Primary key on **Id** with **ValueGeneratedNever()**.

- Required constraints and maximum length constraints on Name (100 characters) and Color (50 characters).

StationConnectionConfiguration:

- Primary key on Id with **ValueGeneratedNever()**.
- Individual indexes on both FromStationId and ToStationId for efficient lookup in either direction.
- Two separate foreign key relationships to the Station entity (FromStation and ToStation), both with **DeleteBehavior.Restrict**.

PricingRuleConfiguration:

- Primary key on Id with **ValueGeneratedNever()**.

The consistent use of **ValueGeneratedNever()** across all entities is a deliberate design decision driven by the data seeding strategy: all entity identifiers are predefined in JSON seed data files and must remain stable across application restarts.

6.3 Repositories

The Repository pattern abstracts the Entity Framework Core data access logic behind interface-based contracts. Each repository depends on **MetroDbContext** via constructor injection and exposes asynchronous methods.

Repository	Interface	Key Methods
StationRepository	IStationRepository	GetByIdAsync (with Include for Line), GetAllAsync (with Include for Line), GetByLineIdAsync (filtered and ordered), GetAllStationsAsync (projected with AsNoTracking)
StationConnectionRepository	IStationConnectionRepository	GetConnectionsByStationIdAsync, GetAllConnectionsAsync (projected with AsNoTracking)
PricingRuleRepository	IPricingRuleRepository	GetAllAsync (ordered by MinStations)
LineRepository	ILineRepository	GetByIdAsync (with Include for Stations), GetAllAsync

The **GetAllStationsAsync** and **GetAllConnectionsAsync** methods use **AsNoTracking()** combined with **Select()** projection to retrieve only the columns needed for graph construction, minimizing memory usage and query execution time.

6.4 Data Seeding

The **MetroDataSeeder** class initializes the database with data from four JSON files stored in the SeedData directory. The seeding process reads the JSON files, deserializes them using **System.Text.Json** with case-insensitive property matching, and inserts new records into the database. Duplicate detection is performed by

querying existing identifiers and filtering out any records that already exist, ensuring idempotent seeding across multiple application restarts.

The seeding sequence respects foreign key dependencies: lines are seeded first, followed by stations (which depend on lines), then connections (which depend on stations), and finally pricing rules. Each seeding step is wrapped in an independent try-catch block, preventing a failure in one step from blocking subsequent steps.

7. Business Logic Layer

7.1 Graph Construction (GraphBuilder)

The **GraphBuilder** service constructs a weighted, undirected graph represented as an adjacency dictionary with the signature **Dictionary<int, List<Neighbor>>**, where the key is a station identifier and the value is a list of neighboring stations with a uniform weight of 1.

```
public async Task<Dictionary<int, List<Neighbor>>> BuildGraphAsync()
{
    if (await _cache.TryGetValue(CacheKey, out var cachedGraph) && cachedGraph is not null)
        return cachedGraph;

    // Fetch all connections from the repository
    var connections = await _stationConnectionRepository.GetAllConnectionsAsync();

    // Initialize the graph structure
    var graph = new Dictionary<int, List<Neighbor>>();
    var edgeSet = new HashSet<string>();

    // Build the graph by processing connections
    foreach (var connection in connections)
    {
        // Add edges to the graph, ensuring no duplicates
        AddEdge(graph, edgeSet, connection.ToStationId, connection.FromStationId);
    }

    // Cache the constructed graph
    _cache.Set(CacheKey, graph, TimeSpan.FromMinutes(30));
    return graph;
}
```

Graph construction is performed using projected data (only Id, Name, and LineId for stations; only FromStationId and ToStationId for connections) with AsNoTracking to minimize overhead. A **HashSet** is used to prevent duplicate edges. The completed graph is cached in **IMemoryCache** with a thirty-minute sliding expiration, eliminating the need to rebuild the graph on every request.

7.2 Shortest Path Algorithm (RouteService)

The **RouteService** implements Dijkstra's algorithm using a **PriorityQueue<int, int>** (min-heap) to efficiently select the unvisited station with the smallest distance at each iteration.

```

public async Task<List<int>> GetShortestPathAsync(int startId, int endId)
{
    var graph = await _graphBuilder.BuildGraphAsync();

    // Initialize distances to infinity (int.MaxValue) except for the start station
    var distances = new Dictionary<int, int> { { startId, 0 } };
    var previous = new Dictionary<int, int?> { { startId, null } };

    var priorityQueue = new PriorityQueue<int, int>();

    // Enqueue the start station
    priorityQueue.Enqueue(startId, 0);

    while (priorityQueue.Count > 0)
    {
        priorityQueue.TryDequeue(out var currentStationId, out var currentDistance);

        // If current distance is greater than the known distance, skip
        if (currentDistance > distances[currentStationId])
            continue;

        // If we reached the destination, break
        if (currentStationId == endId)
            break;

        foreach (var neighbor in graph[currentStationId])
        {
            var newDistance = distances[currentStationId] + neighbor.Weight;

            if (newDistance < distances[neighbor.StationId])
            {
                // Update distance and predecessor
                distances[neighbor.StationId] = newDistance;
                previous[neighbor.StationId] = currentStationId;
                priorityQueue.Enqueue(neighbor.StationId, newDistance);
            }
        }
    }

    return ReconstructPath(previous, endId);
}

```

The algorithm initializes all distances to infinity (`int.MaxValue`) except the start station, which is set to zero. At each step, the station with the smallest distance is dequeued. If a shorter path to a neighbor is discovered, the distance and predecessor are updated, and the neighbor is re-enqueued with the new distance. Once the destination is reached, the path is reconstructed by tracing the **previous** dictionary backward from the destination to the source, then reversing the list. Algorithmic complexity is $O((V+E) \log V)$ where V is the number of stations and E is the number of connections.

7.3 Pricing Calculation (PricingService)

The **PricingService** implements a tiered pricing model. The system defines three fare tiers:

Station Count Range	Fare (EGP)
1 — 9 stations	8 EGP
10 — 16 stations	10 EGP
17 or more stations	15 EGP

The service iterates through the pricing rules (ordered by minimum station count) and returns the price of the first rule whose range matches the station count. Input validation ensures the station count is positive and pricing rules are not empty.

7.4 Travel Time Estimation (TravelTimeService)

The **TravelTimeService** calculates estimated travel time using a straightforward formula: journey time equals the number of stations traversed multiplied by two minutes per station. This constant was chosen based on typical dwell times and running intervals in the Cairo Metro system. Input validation rejects negative station counts.

7.5 Transfer Detection (TransferDetectionService)

The **TransferDetectionService** counts the number of line transfers along a path by iterating through the ordered list of stations and comparing each station's `LineId` with the previous station's `LineId`. Each difference indicates a transfer, as passenger must change from one line to another at that point. A path with fewer than two stations produces zero transfers by definition.

```
public int CountTransfers(IList<Station> stations)
{
    if (stations.Count < 2) return 0;

    for (int i = 1; i < stations.Count; i++)
    {
        if (stations[i].LineId != stations[i - 1].LineId)
            transfers++;
    }

    return transfers;
}
```

7.6 Service Orchestration (MetroService)

The **MetroService** serves as the orchestrator, coordinating the five specialist services. Its single public method, **GetRouteAsync**, accepts a start and end station identifier and returns a **RouteResultDto** containing all computed information.

The orchestration workflow proceeds as follows:

1. Input validation: verifies that the start and end stations are different and exist in the database.

2. Route calculation: delegates to **RouteService.GetShortestPathAsync** to obtain the ordered list of station identifiers.
3. Station resolution: iterates through the path identifiers, fetching each station entity and accumulating station names for the result.
4. Travel time estimation: delegates to **TravelTimeService.CalculateTravelTime** with the station count.
5. Transfer detection: delegates to **TransferDetectionService.CountTransfers** with the list of station entities.
6. Pricing: fetches pricing rules and delegates to **PricingService.CalculatePrice**.
7. DTO construction: assembles the **RouteResultDto** with all computed values.

8. Controllers

8.1 RoutesController

File: `Metro/Controllers/RoutesController.cs`

Purpose

The `RoutesController` is the primary controller responsible for handling route search requests. It manages the complete flow from displaying the search form to computing and presenting route results.

Dependencies

- **IStationRepository** — for retrieving station data to populate dropdown lists.
- **IMetroService** — for delegating route computation to the service layer.

Actions

GET /Routes/Index

- Creates a new **RouteSearchViewModel** instance.
- Calls **LoadStationsAsync()** to populate the Stations list with all available stations mapped to **StationOptionViewModel** objects.
- Returns the Index view with the initialized view model, presenting the user with an empty search form.

POST /Routes/Index

- Receives the submitted **RouteSearchViewModel** through model binding.
- Performs manual validation: checks that both `FromStationId` and `ToStationId` are selected and different from each other.
- On validation failure, sets an error message on the view model and re-renders the form.
- On validation success, calls **_metroService.GetRouteAsync(fromId, toId)** to compute the route.
- On successful computation, assigns the result to the view model and re-renders the view.
- On exception, captures the error message and re-renders the form.
- In all cases, reloads the Stations list to ensure dropdowns are populated.

Request Flow

GET request: Browser → `RoutesController.Index (GET)` → `LoadStationsAsync` → `StationRepository.GetAllAsync` → `MetroDbContext` → `SQL Server` → Station list → `ViewModel` → View (form rendered).

POST request: Browser form submission → `[ValidateAntiForgeryToken]` → Model binding → Manual validation → `MetroService.GetRouteAsync` → `RouteService` → `GraphBuilder` → `Dijkstra algorithm` → `ResultDto` → `ViewModel` → View (result displayed).

8.2 HomeController

File: `Metro/Controllers/HomeController.cs`

Purpose

The HomeController serves as the landing page controller and contains an earlier implementation of route search functionality. It also provides supporting pages for privacy policy and error handling.

Dependencies

- **IStationRepository** — for station data.
- **IRouteService** — for direct route calculation (circumventing the orchestrator).
- **IPricingRuleRepository** — for direct fare calculation.

Actions

- **GET /Home/Index** — Loads stations via ViewBag and returns the home view.
- **POST /Home/Index** — Accepts fromStationId and toStationId as primitive parameters, performs manual validation, calculates route directly through IRouteService, computes fare by querying pricing rules directly, and stores results in ViewBag properties.
- **GET /Home/Privacy** — Returns the privacy policy view.
- **GET /Home/Error** — Returns the error view with the current request identifier, decorated with **[ResponseCache(NoStore = true)]**.

9. Views and User Interface

9.1 ViewModels

RouteSearchViewModel — Serves as the data transfer object between the RoutesController and its view. Contains:

- **FromStationId** (nullable int) and **ToStationId** (nullable int) — form input values.
- **Stations** (List<StationOptionViewModel>) — data for the two dropdown lists.
- **Result** (object) — populated with a RouteResultDto after successful computation, kept as object for view-level casting.
- **ErrorMessage** (string) — validation or service error message.

StationOptionViewModel — Represents a single station option in the dropdown. Contains Id, Name, LineName, Latitude, Longitude, and a computed **DisplayName** property that formats the station as "Name (LineName)".

9.2 Views

Routes/Index.cshtml — The primary route search view. Includes:

- An HTML form with two Tom Select-enhanced dropdowns for station selection.
- A Leaflet interactive map centered on Cairo (coordinates: 30.0444, 31.2357) at zoom level 11, displaying all stations as markers.
- A result card that appears after successful route computation, showing:
 - Endpoint pills for departure and destination stations.
 - Summary statistics: station count, transfer count, fare, and estimated time.
 - A sequential path list showing all stations along the route.
- A "Use My Location" button that employs the Geolocation API and the Haversine formula to find the nearest station.
- Client-side validation using JavaScript to prevent submission of identical stations.

Home/Index.cshtml — The landing page with similar route search functionality using ViewBag for data transfer. Includes map integration, route display with line-specific color coding, and transfer station highlighting.

Shared/_Layout.cshtml — The shared layout providing the HTML document structure, Bootstrap CSS, site styles with cache-busting (**asp-append-version="true"**), the main content area via **RenderBody()**, a footer, and the Scripts rendering section.

9.3 Client-Side Libraries

Library	Purpose	Integration
Tom Select	Searchable dropdowns	Transforms HTML select elements, allows typing to search among 89 stations, options sorted alphabetically
Leaflet + OpenStreetMap	Interactive map	Renders Cairo metro area map, displays station markers with popups showing station name
Geolocation API	GPS location	Browser API to obtain user's current position, used with Haversine formula to find nearest station
Bootstrap 5	CSS framework	Responsive layout, form styling, utility classes
jQuery Validation	Client-side validation	Validates form input before submission

10. Security

10.1 Anti-Forgery Protection

The RoutesController POST action is decorated with the **[ValidateAntiForgeryToken]** attribute, and the corresponding form includes **@Html.AntiForgeryToken()**. This combination protects against Cross-Site Request Forgery (CSRF) attacks by embedding a unique, server-generated token in the form that is validated on every POST request. The server compares the form token against a cookie token; a mismatch results in a 400 Bad Request response.

10.2 HTTPS Redirection

The middleware pipeline includes **UseHttpsRedirection()**, which automatically redirects all HTTP requests to HTTPS. In production environments, **UseHsts()** (HTTP Strict Transport Security) is enabled, instructing browsers to always connect via HTTPS.

10.3 Error Handling

In non-development environments, the middleware pipeline uses **ExceptionHandler("/Home/Error")** to catch unhandled exceptions and redirect to a dedicated error page. The Error action is decorated with **[ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]** to prevent caching of error responses.

10.4 Input Validation

Input validation is performed manually in the controller layer. The RoutesController validates that both station identifiers are selected, that they are different, and wraps service calls in try-catch blocks to handle exceptions gracefully. Client-side validation complements server-side validation through JavaScript and HTML5 required attributes on select elements.

10.5 Overposting Prevention

The use of strongly-typed ViewModels (RouteSearchViewModel) prevents overposting (mass assignment) attacks. Only the properties defined in the view model are bound from the HTTP request, regardless of additional fields the client may submit.

11. Performance Considerations

11.1 Memory Caching

The metro network graph is cached in **IMemoryCache** with a thirty-minute sliding expiration. The graph is built from database records only on cache miss (first request or after expiration). This eliminates redundant database queries and graph construction overhead on subsequent requests.

11.2 AsNoTracking

All graph-related queries use **AsNoTracking()** to disable Entity Framework Core's change tracker. Since graph data is read-only and never modified, tracking is unnecessary overhead. This reduces memory consumption and improves query execution speed.

11.3 Projection (Select)

Graph construction queries use **Select()** projections to retrieve only the columns required for the graph: station identifiers, names, and line identifiers; connection source and destination identifiers. This minimizes the amount of data transferred from the database server and reduces object allocation.

11.4 Composite Index

A composite database index on (LineId, Order) in the Stations table optimizes queries that filter stations by line and order them by position. This index directly supports the **GetByLineIdAsync** repository method, which includes both a Where filter and an OrderBy clause on these columns.

11.5 DbContext Pooling

The DbContext is registered using **AddDbContextPool** rather than **AddDbContext**. Pooling maintains a pool of reusable DbContext instances, avoiding the overhead of creating and disposing a new context for each request. This is particularly beneficial under concurrent load.

11.6 Priority Queue

Dijkstra's algorithm uses **PriorityQueue<int, int>** (introduced in .NET 6) as its min-heap data structure. This provides $O(\log n)$ insertion and $O(\log n)$ extraction of the minimum element, significantly outperforming naive list-based approaches for graph sizes of eighty-nine vertices.

12. Testing and Validation

12.1 Unit Testing Framework

The project includes xUnit (version 2.9.3) and Moq.EntityFrameworkCore (version 8.0.1.7) for unit testing. The test project is configured through the Metro.csproj file with references to xUnit and Moq.

12.2 Validation Scenarios

Scenario	Input	Expected Behavior
Successful route search	Valid station IDs	RouteResultDto with path, price, time, transfers
Missing departure station	FromStationId = null	Error message: "Please select both stations"
Missing destination station	ToStationId = null	Error message: "Please select both stations"
Identical stations	FromStationId = ToStationId	Error message: "Stations cannot be the same"
Inexistent station	FromStationId = 9999	Exception caught, error message displayed
No route exists	Disconnected stations (not possible with current data)	InvalidRouteException, error message displayed
Negative station count	TravelTimeService with -1	ArgumentException thrown
Invalid pricing rule	Station count outside all tiers	InvalidOperationException thrown
Null stations list	TransferDetectionService with null	ArgumentNullException thrown

12.3 Error Handling Strategy

Custom exception classes (**StationNotFoundException**, **InvalidRouteException**) are defined in the Metro.Core project to represent domain-specific error conditions. The MetroService orchestrator catches these exceptions and propagates them to the controller, which displays user-friendly error messages. The Middleware-level exception handler (**UseExceptionHandler**) serves as a safety net for unhandled exceptions.

13. Challenges and Solutions

13.1 Graph Construction from Relational Data

Challenge: Transforming relational database records (stations and connections tables) into an in-memory graph structure suitable for Dijkstra's algorithm required careful mapping and deduplication.

Solution: The GraphBuilder service queries both tables using projected, untracked queries and builds a Dictionary adjacency list. A HashSet-based edge set prevents duplicate entries, and bidirectional edges are created for each undirected connection.

13.2 Dijkstra Algorithm Selection

Challenge: Selecting the optimal shortest-path algorithm for the Cairo Metro network with uniform edge weights.

Solution: Dijkstra's algorithm with a PriorityQueue was chosen over alternatives such as Breadth-First Search (BFS) and A*. BFS would also work for uniform weights, but Dijkstra provides a more general solution that can accommodate weighted graphs in future iterations (e.g., adding travel time or congestion-based weights). A* would require a heuristic function, which adds complexity without significant benefit for this graph size.

13.3 Memory Caching Strategy

Challenge: The metro network graph is static data that is expensive to rebuild on every request, but must be refreshed periodically to reflect potential data changes.

Solution: IMemoryCache with a thirty-minute sliding expiration provides an optimal balance between performance and data freshness. The graph is built once and reused for all requests within the cache window.

13.4 Data Seeding Idempotency

Challenge: Running the database seeder on every application start would cause duplicate key violations if data already exists.

Solution: Each seeding step queries existing identifiers before inserting new records, filtering out any that already exist. This ensures idempotent execution across multiple application restarts.

13.5 Transfer Detection Logic

Challenge: Accurately detecting line transfers along a computed path without external schedule data.

Solution: The line transfer detection algorithm compares consecutive station LineId values. A change in LineId indicates a transfer. This approach is simple, efficient, and works correctly for all path configurations given the Cairo Metro network topology.

13.6 Mixed Arabic and English User Interface

Challenge: The user interface must display station names in Arabic while maintaining English UI elements, requiring careful bidirectional text handling.

Solution: The application uses Unicode bidirectional text with appropriate CSS direction properties. Station data includes Arabic names in the JSON seed files, and the Leaflet map displays station names correctly regardless of script direction.

14. Future Enhancements

The following enhancements are proposed for future iterations of the Cairo Metro Route Planner:

1. **Real-Time Train Schedules:** Integrate with live train position data to provide departure times, waiting times, and real-time journey duration estimates.
2. **Mobile Application:** Develop a cross-platform mobile application using .NET MAUI or React Native to provide native mobile access with offline capabilities.
3. **Multi-Criteria Route Optimization:** Extend Dijkstra's algorithm to support multiple optimization criteria, allowing users to choose between fastest route, fewest transfers, or lowest fare.
4. **User Authentication and Personalization:** Add user accounts to enable saving favorite routes, travel history, and personalized settings.
5. **Accessibility Information:** Include station accessibility data (elevators, escalators, wheelchair access) and provide accessible route options.
6. **Multi-Language Support:** Expand beyond English and Arabic to support additional languages, broadening accessibility for tourists.
7. **Performance Metrics Dashboard:** Add an administrative dashboard with usage statistics, popular routes, and system performance metrics.
8. **API Endpoints:** Expose RESTful API endpoints for third-party integration, enabling other applications to consume the route calculation service.
9. **Automated Testing Coverage:** Expand unit test coverage to include all service classes, repository methods, and controller actions, and implement integration tests against a test database.
10. **Crowdsourced Data:** Allow users to report service disruptions, station closures, and other real-time events that affect route planning.

15. Conclusion

15.1 Project Achievements

The Cairo Metro Route Planner successfully demonstrates the application of software engineering principles to a real-world transportation problem. The system delivers accurate shortest path computation, fare estimation, travel time prediction, and transfer detection for the Cairo Metro network through an intuitive web interface.

15.2 Technical Outcomes

The project achieved the following technical outcomes:

- Implementation of Dijkstra's algorithm with PriorityQueue-based optimization for efficient shortest path computation.
- Design and implementation of a three-tier architecture with clear separation of concerns across presentation, business logic, and data access layers.
- Application of the Repository pattern, Dependency Injection, and the Service Layer pattern to achieve loose coupling and high testability.
- Integration of multiple client-side libraries (Tom Select, Leaflet, Bootstrap) to deliver a rich, interactive user experience.
- Use of Entity Framework Core with Fluent API configurations, JSON-based data seeding, and performance optimization techniques including caching, projection, and untracked queries.

15.3 Lessons Learned

Throughout the development process, several key lessons were identified:

- The importance of designing a flexible graph data structure that can accommodate future extensions beyond uniform edge weights.
- The value of caching strategies in web applications for data that is expensive to compute but changes infrequently.
- The trade-offs between the simplicity of ViewBag-based data transfer and the type safety of strongly-typed ViewModels.
- The critical role of idempotent data seeding in applications that require initial data loading.
- The effectiveness of the orchestrator pattern in decomposing complex business workflows into focused, single-responsibility services.

15.4 Closing Remarks

The Cairo Metro Route Planner stands as a practical, functional application that addresses a genuine need for metro passengers in Cairo. It combines sound software architecture with an accessible user interface,

demonstrating proficiency in ASP.NET Core MVC, Entity Framework Core, algorithm implementation, and modern web development practices. The project provides a solid foundation for future enhancements and real-world deployment.